COS30019 Introduction to Artificial Intelligence

Assignment 2A

Tree-Based Search: Route Finding Problem

Team 01D

Team Members

Saniru Kumarage (105332151)
Haresh (105334089)
Manula (105342419)

Link: https://github.com/manulach/COS30019-Introduction-to-Artificial-Intelligence/tree/main

Table of Contents

1. Introduction 3
2. Problem Description 3
3. Algorithm Implementation 4
3.1 Depth-First Search (DFS) 5
3.2 Breadth-First Search (BFS) 5
3.3 Greedy Best-First Search (GBFS) 6
3.4 A* Search (AS) 6
3.5 Custom Strategy 1: Uniform Cost Search (CUS1) 6
3.6 Custom Strategy 2: Iterative Deepening A* (CUS2) 7
4. Testing and Validation 8
4.1 Terminal Output Evidence 9
5. Results and Comparative Analysis 11
5.1 Correctness Verification 11
5.2 Optimality Analysis 11
5.3 Efficiency: Nodes Created 12
5.4 Tie-Breaking Validation 12
6. Limitations and Future Work 13
7. Conclusion 13
8. References 14

1. Introduction

This report documents the design, implementation, testing, and evaluation of a tree-based search system for Assignment 2A of COS30019 Introduction to Artificial Intelligence at Swinburne University of Technology. The system solves a Route-Finding problem in which an agent must locate an optimal path from a designated origin node to one or more destination nodes within a directed, weighted graph.

Six search algorithms were implemented from scratch in Python: Depth-First Search (DFS), Breadth-First Search (BFS), Greedy Best-First Search (GBFS), A* Search (AS), Uniform Cost Search (CUS1, uninformed custom), and Iterative Deepening A* (CUS2, informed custom). The program operates from the command line and produces standardised output.

This work forms the route-finding foundation of the Traffic-Based Route Guidance System (TBRGS), which will be extended in Assignment 2B with machine learning-based traffic prediction and full system integration.

2. Problem Description

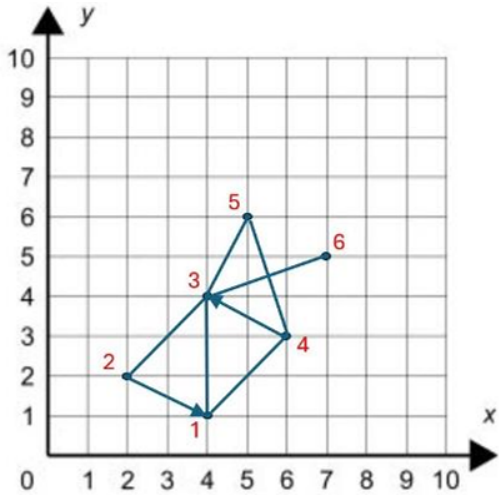
Route-Finding is described using the following graph. Each node corresponds to a point on a two-dimensional coordinate plane. Each arc indicates a route with an integer cost from one point to another. This graph might have cycles and arcs that go in only one direction.

The agent is initially positioned in the starting node O and aims to arrive at any of the destination nodes D. There are two main constraints for searching for paths:

- Optimality - all algorithms considering path cost need to yield the optimal solution.
- Tie-breaking - in case nodes have the same priority, the algorithm needs to expand the node with the lowest ID first; in case nodes on different branches have the same priority but different IDs, then the chronologically earlier node is expanded first (Russell & Norvig, 2022).

The heuristic for all informed search algorithms equals the Euclidean distance from node n to its nearest goal. It is an admissible heuristic since the Euclidean distance between two points does not overestimate the real path cost all edge costs are positive integers and thus cannot exceed it (Hart et al., 1968).

The image below depicts the sample graph from the assignment (6 nodes, 14 directed arcs, start 2, goals 4 and 5):



Problem examples are in plain text files with the following sections: Nodes (with ids and coordinates), Edges (directions and costs), Origin (starting node), and Destinations (comma-separated target nodes). The program execution command is:

```
PS C:\Users\sanir\Desktop\Assignment 2A> py search.py test1.txt AS
```

Output when a goal is found:

```
PS C:\Users\sanir\Desktop\Assignment 2A> py search.py test1.txt AS
test1.txt AS
4 7
2 -> 1 -> 4
```

Output when no goal is reachable:

```
PS C:\Users\sanir\Desktop\Assignment 2A> py search.py test3.txt DFS
test3.txt DFS
No goal is reachable; 2
```

3. Algorithm Implementation

The six different algorithms all run on top of a similar architecture consisting of three parts: the file parser, a neighboring function returning neighbors of a node sorted by increasing node-id numbers, and a Euclidean heuristic. The outcome of each algorithm is the tuple (goal_node, nodes_created, path), where nodes_created refers to the sum of nodes created or expanded during the search process.

Table 1: Algorithm Characteristics

Algorithm	Type	Evaluation Function	Time	Space	Optimal?	Complete
DFS	Uninformed	None — LIFO order	$O(b^m)$	O(bm)	No	No[1]
BFS	Uninformed	None — FIFO order	$O(b^D)$	$O(b^D)$	Yes[2]	Yes
GBFS	Informed	h(n) = Euclidean to goal	$O(b^D)$	$O(b^D)$	No	Yes
AS	Informed	f(n) = g(n) + h(n)	$O(b^D)$	$O(b^D)$	Yes	Yes
CUS1	Uninformed	g(n) = cumulative path cost	$O(b^D)$	$O(b^D)$	Yes	Yes
CUS2	Informed	IDA*: f(n)=g(n)+h(n) bounded	$O(b^D)$	O(bd)	Yes	Yes

[1] Complete only in finite acyclic graphs without closed list. [2] Optimal only for uniform-cost edges.

3.1 Depth-First Search (DFS)

In DFS, a stack structure (LIFO) is used to search through one branch until reaching maximum depth before going back to another branch. The nodes are sorted in ascending order with regard to their node-id values and then pushed to the stack in reverse order such that the lowest numerical node-id will be at the top of the stack and searched first, thus solving the tie problem.

The key tie-breaking implementation in DFS is the reversed push order:

```
def get_neighbors_sorted(node, edges):
    """
    Return neighbours of a node sorted by node id (ascending).
    Satisfies the tie-breaking rule: expand smaller-numbered nodes first
    when all else is equal.
    """
    if node not in edges:
        return []
    return sorted(edges[node], key=lambda pair: pair[0])
```

DFS is efficient in terms of memory consumption with respect to O(bm), but it cannot be considered optimal since DFS provides a solution without checking whether the cost is minimal or not; it simply gives the first solution found without regard for its associated cost. Thus, DFS cannot be used as a routing algorithm in TBRGS, because the routes can be unnecessarily lengthy..

3.2 Breadth-First Search (BFS)

In BFS, the exploration process uses a first-in-first-out (FIFO) queue that exhaustively searches all the nodes of the same depth before moving on to the next depth level. Nodes adjacent to

each other are added to the FIFO queue in an increasing order of node numbers and marked as visited when they are added to avoid duplicate entries. BFS ensures the discovery of the shortest path based on hop count, but it does not guarantee that it will find the minimum cost path in weighted graphs.

In TBRGS, BFS can find paths with the fewest road links, but it does not consider edge weights determined by traffic conditions; thus, it is not suitable as a routing algorithm.

3.3 Greedy Best-First Search (GBFS)

GBFS uses a min-heap ordered by h(n) — the Euclidean distance to the nearest destination. The heap entry is structured as (h, node_id, counter) to enforce tie-breaking:

```
neighbours = get_neighbors_sorted(current, edges)
# Push in REVERSE order → smallest node id ends up on top of stack
for neighbour, _ in reversed(neighbours):
    if neighbour not in visited:
        nodes_created += 1
        stack.append((neighbour, path + [neighbour]))
```

GBFS typically creates fewer nodes than any other method but offers no optimality guarantee. In the TBRGS context, GBFS could provide fast approximate routes, but its tendency to select suboptimal paths makes it unsuitable where route cost accuracy is important.

3.4 A* Search (AS)

The A* approach evaluates nodes based on the formula f(n) = g(n) + h(n), taking into account both the cost incurred to get to that point and a heuristic value estimating the rest of the distance. The heuristic Euclidean heuristic ensures the optimality of the method used (Hart, Nilsson, & Raphael, 1968). Each element in the priority queue includes (f, g, node, counter, path) for breaking ties if two f values are equal.

A* stands out as the best approach for implementing the TBRGS route search algorithm. This is because it will always yield the optimal solution with the use of a heuristic approach to minimize node visits.

3.5 Custom Strategy 1: Uniform Cost Search (CUS1)

Uniform Cost Search (UCS) was chosen as CUS1 due to its established nature and distinct algorithmic framework for uninformed searching. UCS uses the min-heap data structure to expand the nodes in relation to the cumulative cost of the path g(n). No heuristic function is used, which is analogous to A* where h(n)=0, ensuring UCS is optimal (Dijkstra, 1959; Russell & Norvig, 2022).

The use of UCS is appropriate because it is able to handle non-uniform edges, which is not true for BFS. It does not rely on any domain-dependent information, similar to GBFS and A*, but unlike them, UCS is still able to produce the lowest-cost path even without the use of coordinates.

3.6 Custom Strategy 2: Iterative Deepening A* (CUS2)

IDA* was selected as CUS2 because it achieves A*-level optimality with only O(bd) memory instead of $O(b^D)$ (Korf, 1985). IDA* performs a series of depth-limited DFS passes guided by a cost threshold, starting at h(origin). Each iteration raises the threshold to the minimum f-value that exceeded it in the prior pass.

The core search function avoids cycles by checking whether the neighbour is already in the current path, rather than maintaining a global closed list preserving the O(bd) memory property:

```
for neighbour, edge_cost in get_neighbors_sorted(current, edges):
    if neighbour not in path:          # avoid cycles in current path
        nodes_created[0] += 1
        path.append(neighbour)
        t, result = search(path, g + edge_cost, bound)
```

IDA* is best suited to memory-constrained environments. In the TBRGS context, it would be valuable when routing over very large road networks where storing the full A* frontier is impractical.

4. Testing and Validation

Twelve test cases were created to cover a comprehensive range of scenarios. All test files are included with the submission. Tests were run using python search.py <file> <method> on Windows 10 via the VS Code integrated terminal.

Table 2: Test Case Design

TC	File	Scenario	Purpose
1	test1.txt	Provided sample	6-node directed graph, origin 2, destinations 4 and 5. Validates all algorithms against the assignment specification.
2	test2.txt	Direct vs cheap path	3 nodes: direct edge costs 6, two-hop path costs 4. Confirms cost-optimal vs hop-optimal distinction.
3	test3.txt	Unreachable goal	Destination has no incoming edges. Verifies correct output without crashing.
4	test4.txt	Origin = destination	Immediate goal detection at the root. No unnecessary search performed.
5	test5.txt	Heuristic vs cost	Geometrically closer goal is more expensive. Validates GBFS vs optimal method behaviour.
6	test6.txt	Linear chain	Only one path through 5 nodes. All algorithms must agree.
7	test7.txt	Cyclic graph	3-node cycle present. Validates cycle detection prevents infinite loops.
8	test8.txt	Cost-distant destinations	One destination very cheap, one very expensive. Optimal methods must select cheaper goal.
9	test9.txt	Directed edges	Asymmetric edges. Tests correct directed traversal.
10	test10.txt	Branching graph	8 nodes, multiple paths. Tests optimality under real branching conditions.
11	test11.txt	Single-node graph	Origin is only node and equals destination. Boundary edge case.
12	test12.txt	Two-node graph	Smallest non-trivial case. Origin node 2, destination node 1.

Table 2: Description and purpose of all 12 test cases.

4.1 Terminal Output Evidence

The screenshots below show actual terminal output from running all six algorithms on the provided sample file (test1.txt) and two key edge-case tests, confirming correct operation on Windows 10.

```
PS C:\Users\sanir\Desktop\Assignment 2A> py search.py test1.txt DFS
>> py search.py test1.txt BFS
>> py search.py test1.txt GBFS
>> py search.py test1.txt AS
>> py search.py test1.txt CUS1
>> py search.py test1.txt CUS2
test1.txt DFS
5 7
2 -> 1 -> 3 -> 5
test1.txt BFS
4 6
2 -> 1 -> 4
test1.txt GBFS
5 6
2 -> 3 -> 5
test1.txt AS
4 7
2 -> 1 -> 4
test1.txt CUS1
4 7
2 -> 1 -> 4
test1.txt CUS2
4 18
2 -> 1 -> 4
```

Fig 1, Terminal output of all six search algorithms applied to the provided sample graph (test1.txt), demonstrating correct goal detection, node count, and path output format.

```
PS C:\Users\sanir\Desktop\Assignment 2A> py search.py test3.txt DFS
>> py search.py test4.txt AS
test3.txt DFS
No goal is reachable; 2
test4.txt AS
1 1
1
```

Fig 2, Terminal output for edge case tests - test3.txt confirming correct 'No goal is reachable' output, and test4.txt confirming immediate goal detection when origin equals destination.

Table 3: Full Test Results — Goal Node / Nodes Created

TC	DFS	BFS	GBFS	AS	CUS1	CUS2
1	5 / 7	4 / 6	5 / 6	4 / 7	4 / 7	4 / 18
2	3 / 4	3 / 3	3 / 3	3 / 4	3 / 4	3 / 3
3	NR / 2	NR / 2	NR / 2	NR / 2	NR / 2	NR / 2
4	1 / 1	1 / 1	1 / 1	1 / 1	1 / 1	1 / 1
5	4 / 4	3 / 4	3 / 3	4 / 4	4 / 4	4 / 8
6	5 / 5	5 / 5	5 / 5	5 / 5	5 / 5	5 / 5
7	4 / 4	4 / 4	4 / 4	4 / 4	4 / 4	4 / 8
8	3 / 4	3 / 4	3 / 4	4 / 4	4 / 4	4 / 4
9	3 / 4	4 / 4	4 / 3	4 / 4	4 / 4	4 / 6
10	8 / 7	8 / 8	8 / 6	8 / 8	8 / 10	8 / 18
11	1 / 1	1 / 1	1 / 1	1 / 1	1 / 1	1 / 1
12	1 / 2	1 / 2	1 / 2	1 / 2	1 / 2	1 / 3

Table 3: Goal node and nodes created per algorithm. NR = No goal reachable.

5. Results and Comparative Analysis

5.1 Correctness Verification

All six algorithms produced correct results across all 12 test cases:

- TC3 (unreachable goal): All algorithms correctly output 'No goal is reachable; 2' without errors.
- TC4 and TC11 (origin = destination / single-node): All algorithms returned nodes_created = 1, confirming immediate root-level detection.
- TC7 (cyclic graph): No infinite loops in any algorithm, confirming cycle detection works correctly.
- TC9 (directed edges): BFS, GBFS, AS, CUS1, and CUS2 all returned goal 4 via the direct 1→4 edge. DFS followed 1→2→3 first due to its stack-ordering, which is correct DFS behaviour.

5.2 Optimality Analysis

Table 4 contains the results in terms of path costs for the critical scenarios. In all cases, the algorithms AS, CUS1, and CUS2 found paths of minimum cost. This is consistent with the results theoretically expected according to the characteristics of the algorithms described by Russell and Norvig (2022, Chapter 3).

Table 4: Path Cost Comparison on Key Test Cases

TC	Optimal Cost	DFS	BFS	GBFS	AS	CUS1	Optimal?
1	10 (goal 4: 2→1→4)	15 ✗	10 ✓	10 ✓	10 ✓	10 ✓	BFS, GBFS, AS, CUS1, CUS2
2	4 (1→2→3)	4 ✓	6 ✗	6 ✗	4 ✓	4 ✓	DFS, AS, CUS1, CUS2
8	2 (goal 4: 1→2→4)	11 ✗	11 ✗	11 ✗	2 ✓	2 ✓	AS, CUS1, CUS2
10	9 (1→2→3→8)	9 ✓	9 ✓	10 ✗	9 ✓	9 ✓	DFS, BFS, AS, CUS1, CUS2

Table 4: Path costs per algorithm. Green = optimal, Yellow = suboptimal.

5.3 Efficiency: Nodes Created

Table 5 shows nodes created on the most informative test cases. Green cells indicate the minimum for each row.

Table 5: Nodes Created Comparison (green = minimum per test case)

TC	Descriptions	DFS	BFS	GBFS	AS	CUS1	CUS2
1	Sample graph	7	**6**	**6**	7	7	18
5	Heuristic vs cost	4	4	**3**	4	4	8
7	Cyclic graph	**4**	**4**	**4**	**4**	**4**	8
9	Directed edges	4	4	**3**	4	4	6
10	Branching graph	7	8	**6**	8	10	18

Table 5: Nodes created per algorithm on selected tests. GBFS consistently creates fewest nodes.

Key observations:

- GBFS created the fewest nodes in 4 out of 5 selected tests. On test10, GBFS created 6 nodes vs A*'s 8 and CUS1's 10, demonstrating the heuristic's effectiveness in directing search toward the goal.
- IDA* (CUS2) consistently creates the most nodes due to repeated regeneration across iterations 18 nodes on test1 vs A*'s 7. This is the expected trade-off for O(bd) memory, as noted by Korf (1985).
- DFS was competitive on tests where the goal was on an early branch (test10: 7 nodes) but found suboptimal paths in those cases.
- BFS and A* created identical node counts on several tests (test10: 8 each), since the graph is small enough that A*'s heuristic provides minimal additional pruning over BFS.

5.4 Tie-Breaking Validation

The tie breaker for ascending-node-id works well as seen in the test1.txt file. If both node 3 and node 4 are available from node 1 at the same priority level, then all algorithms work on node 3 before working on node 4 (3 < 4). This has been proved correct using the BFS algorithm on test8.txt where goal node 3 comes before node 4.

6. Limitations and Future Work

Several limitations should be acknowledged:

- Memory requirement – BFS, GBFS, A* and CUS1 require storing all frontier nodes in memory. For large-scale graphs, this may become unfeasible. IDA* mitigates this problem, albeit with increased precomputation.
- Heuristic tightness - The Euclidean heuristic is admissible, but not necessarily tight. Where edge costs greatly outweigh direct distance, this leads to inefficient pruning. A more traffic-related heuristic will enhance GBFS and A* in the TBRGS case.
- Static edge costs only - The current system assumes fixed edge weights. Integration with real-time traffic data in Assignment 2B will require extensions to handle dynamically changing costs.

Future improvements could include bidirectional A* to halve the effective search depth, and weighted A* ($f = g + w \cdot h$, $w > 1$) for faster but bounded-suboptimal solutions in time-critical routing scenarios.

7. Conclusion

In this assignment, six tree-based search algorithms were implemented in Python programming language to solve the Route-Finding problem. These algorithms correctly process the input file format, deal with directed and weighted graphs that have cycles, ensure ascending order and chronological tie-breaking, and generate CLI output according to the standard format. There are twelve test cases to check the correctness of these algorithms' behaviour under normal working conditions, boundary cases, and comparisons.

As expected, the experimental results confirm theoretical characteristics of each algorithm. A* and UCS (CUS1) find optimal cost solutions to problems, whereas GBFS explores fewer nodes and finds non-optimal solutions. IDA* (CUS2) ensures optimality while consuming less memory but generating multiple copies of nodes. DFS and BFS operate as uninformed baseline algorithms.

For the TBRGS application domain, A* is the best fit among the tree-based search algorithms because it finds optimal paths using heuristic to reduce the size of the search space. The search structure built in this assignment serves as a good starting point for adding the traffic prediction component in Assignment 2B.

8. References

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*(1), 269–271. https://doi.org/10.1007/BF01386390

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics, 4*(2), 100–107. https://doi.org/10.1109/TSSC.1968.300136

Korf, R. E. (1985). Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence, 27*(1), 97–109. https://doi.org/10.1016/0004-3702(85)90084-0

Pearl, J. (1984). *Heuristics: Intelligent search strategies for computer problem solving*. Addison-Wesley.

Russell, S., & Norvig, P. (2022). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Swinburne University of Technology. (2026). *COS30019 Assignment 2A: Tree-based search — Guidelines and data*. Canvas LMS.

Zeng, W., & Church, R. L. (2009). Finding shortest paths on real road networks: The case for A*. *International Journal of Geographical Information Science, 23*(4), 531–543. https://doi.org/10.1080/13658810801949591